

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	D.Sravanthi	Reg. No.:	192425315
Section / Batch:	2024-AI&ML	Date:	17.08.2026

Code of Conduct

I D.Sravanthi(192425315) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: D.Sravanthi

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Design an algorithm to perform **Reference Resolution** by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

1. Reference Resolution

Algorithm

1. Read the given discourse sentence by sentence.
2. Identify all referring expressions and pronouns.
3. Identify possible antecedents for each pronoun.
4. Check gender and number agreement.
5. Use grammatical role, recency, and discourse context to select the correct antecedent.
6. Replace each pronoun with its corresponding antecedent.
7. Generate the resolved discourse.
8. Validate the result using the identified constraints.

Pseudocode

Algorithm ReferenceResolution(Text)

 Read the input text

 Identify all entities and pronouns

 For each pronoun P:

 Find all possible antecedents before P

 For each candidate antecedent C:

 Check gender agreement

Check number agreement

Check grammatical role

Check recency

Check discourse context

Select the most suitable antecedent C

Replace pronoun P with antecedent C

Return the resolved discourse

End Algorithm

Given Text

"Ravi met Arun at the library. He borrowed a book and later returned it."

Identification of Pronouns and Antecedents

Pronoun	Possible Antecedents	Selected Antecedent
He	Ravi, Arun	Ravi
it	book	book

Resolved Discourse

"Ravi met Arun at the library. Ravi borrowed a book and later returned the book."

Explanation and Validation

The pronoun **"He"** can refer to either Ravi or Arun because both are male. However, **Ravi** is selected based on discourse context and grammatical prominence because Ravi is the subject of the first sentence and is the most natural continuing topic.

The pronoun **"it"** refers to **"a book"** because *it* is singular and the book is the object that can logically be returned.

Therefore, the resolution is:

He → Ravi

it → Book

The algorithm uses **gender, number, grammatical role, recency, and discourse context** to identify the most appropriate antecedents and produce a coherent resolved discourse.

2. Develop an algorithm to analyze **Text Coherence** and **Discourse Structure** by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

2. Text Coherence and Discourse Structure

Algorithm

1. Read the given discourse.
2. Divide the discourse into individual sentences.
3. Identify the main events or ideas in each sentence.
4. Detect discourse markers such as **therefore, because, after, then**, etc.
5. Identify the logical relationship between the sentences.
6. Classify the relationships as **cause-effect, sequence, elaboration**, or other suitable relations.
7. Construct a discourse structure connecting the sentences.
8. Check whether the identified relationships provide a logical and coherent flow.

Pseudocode

Algorithm AnalyzeTextCoherence(Text)

Split Text into sentences

For each sentence:

Identify main events and concepts

Identify discourse markers

For each pair of related sentences:

If one event causes another event:

Relation = CAUSE-EFFECT

Else if one event occurs after another:

Relation = SEQUENCE

Else if one sentence provides additional information:

Relation = ELABORATION

Connect the sentences using the identified relation

Return discourse structure

End Algorithm

Given Discourse

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Discourse Relations

Sentence 1:

"The roads were flooded after heavy rainfall."

This establishes the **cause and situation**. Heavy rainfall caused the roads to become flooded.

Sentence 2:

"Therefore, schools were closed for the day."

This has a **Cause-Effect** relationship with Sentence 1. The flooding resulted in schools being closed.

Sentence 3:

"Students attended classes online."

This has a **Consequence/Sequence** relationship with Sentence 2. Since schools were closed, students attended classes online.

Discourse Structure

Heavy Rainfall



Roads Were Flooded



Schools Were Closed



Students Attended Classes Online

Relation Representation

S1: Roads were flooded

CAUSE-EFFECT



S2: Schools were closed

CONSEQUENCE / SEQUENCE



S3: Students attended classes online

Validation

The discourse is coherent because each sentence is logically connected to the next. The word **"Therefore"** explicitly indicates a cause-effect relationship between the first and second sentences. The third sentence describes the consequence of school closure.

Thus, the overall structure follows:

Cause → Effect → Consequence

These relationships maintain **text coherence** by creating a clear logical flow and helping the reader understand how the events are connected.

3. Design an algorithm for a **Conversational Agent** that identifies **Dialogue Acts** from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?"

Agent: "Sure, where would you like to travel?"

User: "I want to go to Chennai."

Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

3. Conversational Agent – Dialogue Act Identification

Algorithm

1. Read each utterance in the conversation.
2. Identify important keywords and sentence patterns.
3. Determine the intention of each utterance.
4. Classify the utterance as **Request, Question, Inform, Confirmation, or Action**.
5. Store the identified dialogue acts in sequence.
6. Use the dialogue-act sequence to understand the user's intention.
7. Generate the appropriate response or action.

Pseudocode

Algorithm IdentifyDialogueActs(Conversation)

For each utterance U in Conversation:

If U asks the agent to perform an action:

Act = REQUEST

Else if U asks for information:

Act = QUESTION

Else if U provides information:

Act = INFORM

Else if U confirms completion:

Act = CONFIRMATION

Else if U performs an operation:

Act = ACTION

Store Act

Return dialogue-act sequence

End Algorithm

Given Conversation

User: "Can you book a train ticket for me?"

Agent: "Sure, where would you like to travel?"

User: "I want to go to Chennai."

Agent: "Your ticket has been booked."

Dialogue Act Classification

Speaker	Utterance	Dialogue Act
User	"Can you book a train ticket for me?"	Request

Speaker	Utterance	Dialogue Act
Agent	"Sure, where would you like to travel?"	Question
User	"I want to go to Chennai."	Inform
Agent	"Your ticket has been booked."	Confirmation

Dialogue-Act Sequence

Request → Question → Inform → Confirmation

Validation

The first utterance is a **Request** because the user asks the agent to book a train ticket. The agent asks a **Question** to obtain the required destination. The user provides the destination through an **Inform** act. Finally, the agent gives a **Confirmation** that the requested booking has been completed.

Thus, dialogue acts help the conversational agent understand the user's intention, maintain the conversation context, collect missing information, and perform the requested task correctly.

4. Design an algorithm for **Language Generation** that converts a semantic representation into a grammatically correct sentence through **Surface Realization**. Consider the semantic input:

Action: Buy

Agent: Student

Object: Book

Tense: Past

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

4. Language Generation through Surface Realization

Semantic Input

Action: Buy

Agent: Student

Object: Book

Tense: Past

Algorithm

1. Read the semantic representation.
2. Identify the **Agent, Action, Object, and Tense**.
3. Perform lexical selection by choosing suitable words.
4. Arrange the words using the **Subject–Verb–Object (SVO)** structure.
5. Convert the verb into the required tense.
6. Add appropriate articles and grammatical markers.
7. Apply surface realization to generate the final sentence.
8. Check the generated sentence for grammatical correctness.

Pseudocode

Algorithm SurfaceRealization(SemanticInput)

 Extract Agent, Action, Object, and Tense

 Select suitable lexical items

 Arrange as:

 Subject + Verb + Object

 If Tense = Past:

 Convert Action to past tense

 Add articles and punctuation

 Generate final sentence

 Validate grammatical correctness

 Return sentence

End Algorithm

Application

Semantic Representation:

- Agent → Student
- Action → Buy
- Object → Book
- Tense → Past

Lexical Selection:

- Student → **The student**
- Buy → **bought**
- Book → **a book**

Sentence Structure:

Subject + Verb + Object

The student + bought + a book.

Final Natural Language Sentence

"The student bought a book."

Explanation

The algorithm first performs **lexical selection** by converting the semantic concepts into appropriate English words. The agent *Student* becomes **the student**, the action *Buy* is converted to its past-tense form **bought**, and the object *Book* becomes **a book**.

Next, the algorithm performs **sentence structuring** using the Subject–Verb–Object pattern. Finally, **surface realization** applies grammatical rules, including the correct past-tense verb, article usage, capitalization, and punctuation.

Validation

The generated sentence **"The student bought a book."** is grammatically correct because it contains:

- **Subject:** The student
- **Verb:** bought

- **Object:** a book
- **Tense:** Past

Therefore, the semantic representation is successfully converted into a grammatically correct natural-language sentence.

5. Design an algorithm that combines **Interlingua-based Machine Translation** and **Statistical Translation Approaches**. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. Analyze the source sentence.
2. Create an Interlingua representation.
3. Generate candidate target-language translations.
4. Apply statistical scoring to select the most appropriate translation.
5. Produce the final translated sentence.

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

5. Interlingua-Based and Statistical Machine Translation

Algorithm

1. Read and analyze the source sentence.
2. Identify the subject, verb, object, tense, and aspect.
3. Convert the meaning into an **Interlingua representation**.
4. Generate multiple candidate translations in the target language.
5. Calculate a statistical score for each candidate.
6. Select the candidate with the highest score.
7. Generate the final translated sentence.
8. Validate the meaning and grammatical correctness.

Pseudocode

Algorithm HybridMachineTranslation(SourceSentence)

Analyze SourceSentence

Identify Subject, Verb, Object, Tense and Aspect

Create Interlingua representation

Generate candidate target translations

For each candidate:

 Calculate Translation Probability

 Calculate Language Model Probability

 Calculate Overall Score

Select candidate with highest score

Return final translation

End Algorithm

Step 1: Analyze the Source Sentence

Source Sentence:

"The boy is playing football."

The sentence contains:

- **Subject:** The boy
- **Verb:** is playing
- **Object:** football
- **Tense:** Present
- **Aspect:** Progressive

The meaning is that the boy is currently engaged in playing football.

Step 2: Interlingua Representation

The sentence can be represented using a language-independent semantic structure:

PLAY

AGENT = BOY

OBJECT = FOOTBALL

TENSE = PRESENT

ASPECT = PROGRESSIVE

This representation stores the meaning of the sentence without depending on the grammar of a particular language.

Step 3: Generate Candidate Target-Language Translations

For example, for Tamil, possible candidates are:

1. சிறுவன் கால்பந்து விளையாடுகிறான்.
2. சிறுவன் கால்பந்து விளையாடினார்.
3. சிறுவன் கால்பந்து விளையாடுவான்.

Step 4: Statistical Scoring

The system calculates a score for each candidate using translation probability and language-model probability.

Overall Score =

Translation Probability $\times$ Language Model Probability

The first candidate receives the highest score because it correctly represents the **present progressive action** and is grammatically natural in Tamil.

Step 5: Final Translation

சிறுவன் கால்பந்து விளையாடுகிறான்.

Evaluation

The **Interlingua representation** captures the basic meaning of the source sentence independently of any particular language. It identifies the agent, action, object, tense, and aspect, thereby reducing ambiguity.

The **statistical translation method** compares possible target-language translations and selects the most probable and grammatically natural output.

Therefore, combining Interlingua with statistical scoring improves translation quality by providing **semantic accuracy, reduced ambiguity, better candidate selection, and more natural target-language output.**

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____